

CUSTOM SOCKET DESIGN (KTP)

1. Proper Error Handling and Resource Management:
 - a. The code consistently checks the return values of system calls, such as `shmget`, `semget`, `socket`, and `bind`, and handles errors appropriately using `perror` and setting `errno`.
 - b. The `sigHandler` function ensures resources (shared memory and semaphores) are properly released on program termination (e.g., via `Ctrl+C`), preventing resource leaks.
2. Thread-Safe Shared Memory Access:
 - a. The code uses semaphores (`sem_SM`, `sem_sock_info`) to protect access to shared memory structures (`SM` and `sock_info`), ensuring thread-safe operations.
 - b. Critical sections are marked with `P(sem)` and `V(sem)` operations to prevent race conditions.
3. Efficient Use of System Calls and Timeouts:
 - a. The `select` system call is used in the `R` thread to efficiently monitor multiple UDP sockets with a timeout, reducing busy waiting.
 - b. The `S` thread uses `sleep(T/2)` to periodically check for timeouts and retransmit unacknowledged packets, ensuring reliable message delivery.
4. Modular and Well-Defined Protocol Implementation:
 - a. The code implements a custom protocol (`SOCK_KTP`) on top of UDP, with clear header formats for ACK and DATA messages (9-bit header with type and sequence number).
 - b. Functions like `k_socket`, `k_bind`, `k_sendto`, and `k_recvfrom` provide a socket-like API, making it easy to test and integrate with existing applications

GitHub

Link: [https://github.com/adhi0987/Computer_Networks/tree/main/CustomSocket_\(KTP\)](https://github.com/adhi0987/Computer_Networks/tree/main/CustomSocket_(KTP))

Screenshots:

```
adhithya@pop-os: ~/Desktop/networks_lab_24-25/KTP_SOCKET$ ./user2 127.0.0.1 5000
127.0.0.1 6000
Reading file
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
```

```
adhithya@pop-os: ~/Desktop/networks_lab_24-25/KTP_SOCKET$ ./user1 0.0.0.0 6000 127.0.0.1 5000
Reading file
Sending 1024 bytes
Sent 1024 bytes, seq 1
Sending 1024 bytes
Sent 1024 bytes, seq 2
Sending 1024 bytes
Sent 1024 bytes, seq 3
Sending 1024 bytes
Sent 1024 bytes, seq 4
Sending 1024 bytes
Sent 1024 bytes, seq 5
Sending 1024 bytes
Sent 1024 bytes, seq 6
Sending 1024 bytes
Sent 1024 bytes, seq 7
Sending 1024 bytes
Sent 1024 bytes, seq 8
Sending 1024 bytes
Sent 1024 bytes, seq 9
Sending 1024 bytes
Sent 1024 bytes, seq 10
Sending 1024 bytes
Sent 1024 bytes, seq 11
```

```
Sent 1024 bytes, seq 6
Sending 1024 bytes
Sent 1024 bytes, seq 7
Sending 1024 bytes
Sent 1024 bytes, seq 8
Sending 1024 bytes
Sent 1024 bytes, seq 9
Sending 1024 bytes
Sent 1024 bytes, seq 10
Sending 1024 bytes
Sent 1024 bytes, seq 11
Sending 1024 bytes
Sent 1024 bytes, seq 12
Sending 1024 bytes
Sent 1024 bytes, seq 13
Sending 1024 bytes
Sent 1024 bytes, seq 14
Sending 1024 bytes
Sent 1024 bytes, seq 15
Sending 310 bytes
Sent 310 bytes, seq 0
Sending EOF. No. of messages sent = 113
File sent.
```

```
adhithya@pop-os: ~/Desktop/networks_lab_24-25/KTP_SOCKET$
```

```
Writing 1023 bytes
Writing 1023 bytes
Writing 1023 bytes
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Writing 1023 bytes
Writing 1023 bytes
Writing 1023 bytes
Writing 1023 bytes
Writing 1023 bytes
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Checking recv
Writing 1023 bytes
Writing 1023 bytes
Writing 309 bytes
End of file
```